

学业发展规划系统 - 项目展示演讲稿

使用说明

- 【讲解提示】标记的内容为演讲技巧提示，不会出现在正式演讲中，请在准备时参考
- "技术细节"部分需要配合代码截图或现场演示效果更佳
- 预计演讲时长：15-20 分钟

开场白

尊敬的各位老师、同学们，大家好！

我是XXX，今天非常荣幸能够向大家展示我的期末大作业——**学业发展规划系统**。

这是一个基于 **Qt 6** 框架和 **C++17** 开发的跨平台桌面应用程序，采用分层架构设计，总计约 **15,200 行代码**、**151 个源文件**，旨在帮助学生系统地规划和管理自己的学业发展。

接下来，我将从项目背景、架构设计、核心功能的技术实现、难点攻克和项目亮点五个方面进行详细汇报。

【讲解提示】开场白控制在 30 秒内，语速适中。提到"15,200 行代码"时可以停顿一下，让评委注意到项目规模。可以用手示意屏幕上的系统截图作为视觉引导。

一、项目背景与需求分析

1.1 项目背景

在当今高等教育环境中，学生面临着越来越多的学业挑战：

- 课程管理复杂**：需要管理多门课程的成绩、学分、GPA 等信息，数据分散在不同平台
- 目标规划困难**：缺乏系统化工具来制定和追踪学业目标
- 经历记录分散**：实习、项目、竞赛等经历没有统一的归档方式
- 简历制作繁琐**：每次写简历都需要从头整理信息，重复劳动
- 数据分析不足**：缺乏对自身学业数据的深度分析，无法直观看到进步趋势

1.2 需求分析

基于以上痛点，我确定了以下核心需求：

- 1. **全面的数据管理**：支持课程、角色、成果、经历、活动、目标、岗位等 7 类数据的增删改查
- 2. **智能数据分析**：提供 GPA 趋势分析、学期对比、同学对标等可视化功能
- 3. **简历自动生成**：基于已有数据智能生成简历，支持多格式导出
- 4. **AI 辅助分析**：通过规则引擎和远程 AI 提供学业建议
- 5. **数据导入导出**：支持 CSV 批量导入
- 6. **现代化界面**：Material Design 风格，流畅动画

【讲解提示】这一部分讲 1-2 分钟即可，重点是让评委理解"为什么要做这个系统"。可以举一个具体场景，比如："我曾经为了写简历，要翻遍手机相册、各个平台的记录去拼凑经历，这就是我做这个系统的初衷。"

二、技术选型与架构设计

2.1 技术栈选择

技术	版本	选择理由
Qt	6.5+	跨平台能力强，内置 UI 组件、网络、数据库、HTTP Server 等模块
C++	17	高性能，结构化绑定、 <code>std::function</code> 、 <code>constexpr</code> 等现代特性
SQLite	3.x	嵌入式数据库，通过 <code>QSqlDatabase</code> / <code>QSqlQuery</code> 直接访问，零配置部署
CMake	3.25+	现代化构建系统，跨平台支持好

【讲解提示】技术选型部分要体现你的思考过程，而不是简单罗列。可以强调："选择 Qt 6 是因为它在跨平台框架中对 C++17 的支持最好，且内置了完善的数据库和网络模块。"这体现了你对技术的深入理解。

2.2 系统架构

系统采用**六层分层架构**，从上到下依次为：

表示层 (Presentation Layer)
MainWindow → QStackedWidget → 12个Page + 9个Dialog
布局管理: QHBoxLayout / QVBoxLayout / QGridLayout / QSplitter
核心层 (Core Layer)
AppShellController - 壳层控制 (QStackedWidget 页切换)
DataRefreshCoordinator - 刷新协调 (信号槽跨页面同步)
BackendRuntimeController - 后端管理 (QProcess 启停)
AiContextMediator - AI上下文 (QEvent::eventFilter 事件过滤)
业务层 (Business Layer)
Service 层: 13个服务类 (CourseService, AiService 等)
Model 层: 11个数据模型 (Course, Goal, Role 等)
数据层 (Data Layer)
DAO 层: 继承 DaoBase → QSqlDatabase + QSqlQuery
SQLite 数据库: schema.sql 定义表结构

2.3 核心协作关系：MainWindow 如何"指挥"各模块

这是整个系统最核心的设计。MainWindow 本身只有 **766 行**（从最初的 5000+ 行重构而来），它不直接处理业务逻辑，而是通过四个**核心协调器**来分发职责：

```
MainWindow (QMainWindow)
|
|— m_shellController (AppShellController*) — 管理 QStackedWidget 页面切换
|   |— 接收 NavigationListWidget 的 currentRowChanged 信号
|   |   → 更新 topbar 标题、内容宽度、间距
|
|— m_refreshCoordinator (DataRefreshCoordinator*) — 管理跨页面数据刷新
|   |— bindPages() 绑定全部 12 个页面
|   |   connectSignals() 将每个页面的 dataChanged 信号连接到 refreshByDomain()
|   |   → 实现"课程页改了数据 → 总览页自动刷新GPA"
|
|— m_backendController (BackendRuntimeController*) — 管理 HTTP 后端进程
|   |— 使用 QProcess 启动/停止后端服务
|   |   bindWidgets() 绑定 QProgressBar + QLabel + QSystemTrayIcon
|   |   → 通过信号 backendReady / serverError 通知 MainWindow
|
|— m_aiMediator (AiContextMediator*) — 管理 AI 上下文传递
|   |— bindRootWidget() 在根 Widget 上安装 eventFilter
|   |   重写 eventFilter() 拦截 QEvent::MouseButtonRelease
|   |   → 检测用户选中文本 → 弹出浮窗菜单 → 发送到 AiPanelWidget
```

页面注册机制：MainWindow 使用 `std::function` 注册表替代了传统的 `switch-case` 硬编码：

```
// 将每个页面添加到 QStackedWidget，并注册其刷新回调
m_overviewPage = new OverviewPage(this);
m_pageRefreshers[m_stack->addWidget(m_overviewPage)] = [this]() {
    m_overviewPage->refresh();
};
// ... 12 个页面依次注册
```

这样在切换页面时，只需一行代码即可刷新：

```
void MainWindow::refreshCurrentPage() {
    int index = m_stack->currentIndex();
    if (m_pageRefreshers.contains(index)) {
        m_pageRefreshers[index](); // 调用对应页面的 refresh()
    }
}
```

【讲解提示】这是演讲的核心亮点之一。建议打开架构图的 PPT 页面，用手指着图讲解："MainWindow 就像一个'指挥官'，它不亲自干活，而是把任务分给四个协调器。比如用户在课程页修改了成绩，DataRefreshCoordinator 会自动通知总览页刷新 GPA 显示——这就是 Qt 信号槽机制的典型应用。" **这里可以现场演示：先切到课程页加一门课，再切回总览页看 GPA 变化。**

三、核心功能的技术实现

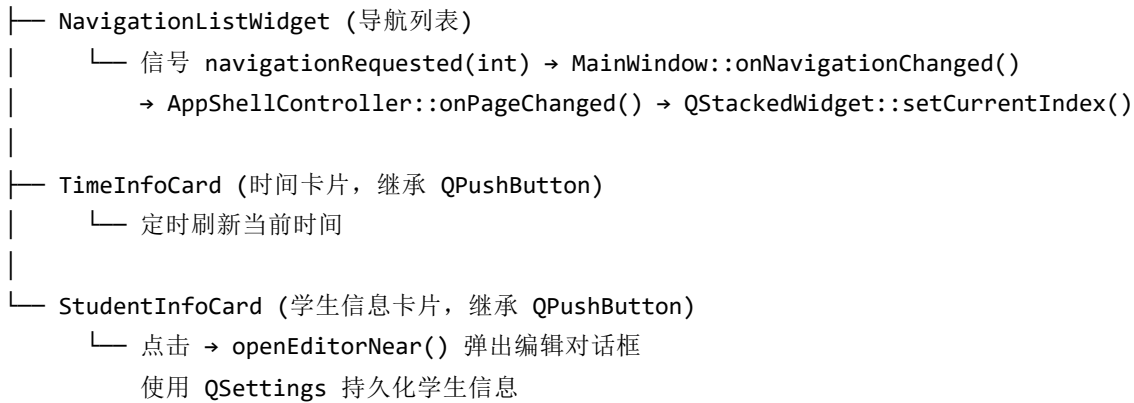
3.1 侧边栏导航与动画系统

涉及的 Qt 类：

- SidebarWidget (继承 QFrame) — 侧边栏容器
- NavigationListWidget (继承 QListWidget) — 导航列表
- QPropertyAnimation + QParallelAnimationGroup — 并行动画
- QEasingCurve::InOutQuad — 缓动曲线
- TimeInfoCard / StudentInfoCard (继承 QPushButton) — 信息卡片

类之间的协作：

SidebarWidget



侧边栏折叠动画的关键实现：

```
void SidebarWidget::setupToggleAnimation() {
    connect(m_toggleButton, &QPushButton::clicked, this, [this]() {
        auto* group = new QParallelAnimationGroup(this);
        auto* anim1 = new QPropertyAnimation(this, "minimumWidth");
        auto* anim2 = new QPropertyAnimation(this, "maximumWidth");
        anim1->setDuration(220);
        anim1->setEasingCurve(QEasingCurve::InOutQuad);
        // ... 同时对 minimumWidth 和 maximumWidth 做动画
        // 折叠时隐藏 brandMark 和 brandText
        // 展开时重新显示
        group->start();
    });
}
```

使用 `QParallelAnimationGroup` 让 `minimumWidth` 和 `maximumWidth` 两个属性**同时**做动画，确保宽度平滑过渡。`QEasingCurve::InOutQuad` 提供了"先慢后快再慢"的自然缓动感。

【讲解提示】侧边栏动画是一个很好的视觉演示点。可以现场点击折叠按钮，让评委看到流畅的动画效果。讲解时可以说："这里用了 Qt 的属性动画系统，`QPropertyAnimation` 可以对任何 `QObject` 的属性做动画，配合 `QParallelAnimationGroup` 实现多属性同步。"

3.2 数据管理：CRUD 页面的统一架构

涉及的 Qt 类：

- `BasePage` (继承 `QWidget`) — 页面基类，定义纯虚函数 `refresh()`
- `CrudPageController` (继承 `QObject`) — 通用 CRUD 控制器
- `QTableWidget` / `QListWidget` — 数据列表展示
- `QDialog` 子类 (如 `GoalEditorDialog`、`JobEditorDialog`) — 编辑对话框
- `QPushButton` + `QLineEdit` + `QComboBox` + `QDoubleSpinBox` + `QPlainTextEdit` — 表单控件

CrudPageController 的策略模式设计：

这是项目中一个很巧妙的设计。CrudPageController 使用 std::function 回调实现了一个**通用的 CRUD 框架**，任何页面都可以复用：

```
class CrudPageController : public QObject {
    // 策略函数类型定义
    using CreateDialogFn = std::function<QJsonObject(QWidget*)>;
    using EditDialogFn   = std::function<QJsonObject(QWidget*, const QJsonObject*)>;
    using ServiceCreateFn = std::function<void(const QJsonObject*)>;
    using ServiceUpdateFn = std::function<void(int, const QJsonObject*)>;
    using ServiceRemoveFn = std::function<void(int)>;
    // ...

    void executeAdd(QWidget* parent);    // 弹出创建对话框 → 调用 Service 创建
    void executeEdit(QWidget* parent, int id); // 弹出编辑对话框 → 调用 Service 更新
    void executeRemove(QWidget* parent, int id); // 确认删除 → 调用 Service 删除
};
```

以目标页面为例的完整数据流：

用户点击"添加目标"

- GoalsPage::onAddClicked()
- GoalEditorDialog (QDialog) 弹出
 - ├─ 内含 QLineEdit (标题)
 - ├─ QComboBox (优先级：高/中/低)
 - ├─ QDoubleSpinBox (目标值/当前值)
 - ├─ QPlainTextEdit (描述)
 - └─ validateAndAccept() 校验后返回 QDialog::Accepted
- GoalService::create() 业务逻辑层处理
- GoalDao::create() 数据访问层写入 SQLite
- emit dataChanged(DataDomain::Goals) 发出信号
- DataRefreshCoordinator::refreshByDomain()
- OverviewPage::refresh() 自动更新总览数据

GoalEditorDialog 的控件布局：

```
class GoalEditorDialog : public QDialog {
    QLineEdit* m_titleEdit;           // 标题输入
    QLineEdit* m_categoryEdit;        // 分类输入
    QDoubleSpinBox* m_targetSpin;     // 目标值微调框
    QDoubleSpinBox* m_currentSpin;    // 当前值微调框
    QComboBox* m_priorityCombo;       // 优先级下拉框
    QComboBox* m_statusCombo;         // 状态下拉框
    QPlainTextEdit* m_descriptionEdit; // 多行描述
    // ...
};
```

【讲解提示】这一部分要强调“复用性”。可以说：“我设计了一个通用的 `CrudPageController`，通过 `std::function` 回调注入不同页面的具体逻辑。这样 7 个数据管理页面都共用同一套 CRUD 流程，新增一个页面只需要写对话框和注册回调，核心增删改查代码完全复用。”可以打开两个不同的页面（如课程页和目标页），展示它们的操作流程是一致的。

3.3 数据分析与可视化

涉及的 Qt 类：

- `AnalysisPage` (继承 `BasePage`) — 分析页面
- `SemesterAnalysisWidget` — 学期分析组件
- `PeerBenchmarkWidget` — 同学对标组件
- `MetricGridWidget` (继承 `QFrame`) — 指标网格
- `QGridLayout` — 网格布局排列指标卡片
- `QLabel` — 显示数值指标
- `PeerEditorDialog` (继承 `QDialog`) — 对标同学编辑
- `AnalyticsService` — 分析服务，生成报告数据

`MetricGridWidget` 的设计：

指标网格使用 `QGridLayout` 动态排列指标卡片：

```
struct MetricEntry {
    QString label;           // 指标名称
    QLabel** valueTarget;    // 指向值标签的指针（用于后续更新）
    QString hint;            // 提示文字
};

class MetricGridWidget : public QFrame {
    void setMetrics(const QVector<MetricEntry>& entries, int columns = 4);
    void setValue(int index, const QString& text);
    QVector<QLabel*> m_valueLabels;
};
```

分析页面的数据流：

```
AnalysisPage::refresh()
→ AnalyticsService::generateReport() // 返回 QJsonObject 报告
→ m_semesterWidget->loadReport(report) // 学期分析组件加载数据
→ m_peerWidget->loadPeers(peers)      // 对标组件加载同学数据
→ 更新指标标签文本
```

同学对标功能的信号槽连接：

```
// AnalysisPage 中连接 PeerBenchmarkWidget 的信号
connect(m_peerWidget, &PeerBenchmarkWidget::addPeerRequested,
        this, &AnalysisPage::onAddPeerRequested);
connect(m_peerWidget, &PeerBenchmarkWidget::editPeerRequested,
        this, &AnalysisPage::onEditPeerRequested);
connect(m_peerWidget, &PeerBenchmarkWidget::removePeerRequested,
        this, &AnalysisPage::onRemovePeerRequested);
```

【讲解提示】数据分析页适合配合截图讲解。可以展示"学期分析"和"同学对标"两个子组件的效果图。重点说明："所有数据都通过 `AnalyticsService` 统一计算，页面只负责展示——这就是 MVC 思想的体现。"

3.4 智能简历生成

涉及的 Qt 类：

- `ResumePage` (继承 `BasePage`) — 简历页面
- `ResumeEditorPanel` — 简历编辑面板
- `ResumeCandidatePanel` — 备选素材面板
- `ResumePreviewWidget` / `ResumePreviewDialog` — 简历预览
- `QSplitter` — 可拖拽分割面板
- `QTextEdit` — 富文本编辑
- `QTimer` — 防抖延迟更新预览
- `QClipboard` (通过 `QGuiApplication::clipboard()`) — 剪贴板操作
- `QFileDialog` — 文件保存对话框
- `QDesktopServices::openUrl()` — 打开导出的 HTML 文件
- `QSettings` — 持久化简历配置和头像路径

简历页面的布局结构：

ResumePage

- ├─ QLabel (页面标题)
- ├─ QGridLayout (指标卡片: 简历分区数、身份标题)
- ├─ QHBoxLayout (操作按钮: 预览、刷新、重置、导出JSON/HTML、复制)
- └─ QSplitter (Qt::Horizontal) ← 可拖拽分割
 - ├─ ResumeEditorPanel (左侧编辑区)
 - └─ QTextEdit 富文本编辑器
 - └─ ResumeCandidatePanel (右侧素材推荐区)
 - └─ 从数据库读取经历/成果, 生成推荐素材

防抖机制: 编辑面板的每次修改不会立即触发预览更新, 而是通过 `QTimer` 实现 300ms 防抖:

```
m_updateTimer = new QTimer(this);
m_updateTimer->setSingleShot(true); // 单次触发
m_updateTimer->setInterval(300);     // 300ms 延迟
connect(m_updateTimer, &QTimer::timeout, this, &ResumePage::updatePreview);

// 编辑内容变化时 → 重置计时器
// 只有用户停止输入 300ms 后才真正更新预览
```

多格式导出实现:

- **JSON 导出:** 使用 `QJsonDocument` 序列化, `QFile + QTextStream` 写入
- **HTML 导出:** `buildResumePreviewHtml()` 生成完整 HTML 字符串, 保存为 `.html` 文件
- **复制到剪贴板:** 调用 `QGuiApplication::clipboard()->setText()` 直接复制
- **预览窗口:** `QDesktopServices::openUrl(QUrl::fromLocalFile(path))` 用系统浏览器打开

【讲解提示】简历页面最适合做现场演示。可以: (1) 展示编辑面板和预览的实时联动; (2) 拖拽 `QSplitter` 分割线; (3) 点击"导出 HTML"后用浏览器打开。讲解时强调: "编辑器用 `QTimer` 做了 300ms 防抖, 避免用户每敲一个字就重新渲染预览——这是常见的性能优化手段。"

3.5 AI 辅助系统

这是项目中技术含量最高的模块, 涉及三个核心类的协作。

涉及的 Qt 类:

- `AiPanelWidget` (继承 `QFrame`) — AI 面板组件
- `AiStatusBar` — AI 状态指示器
- `AiConversationWidget` — 对话界面
- `AiContextMediator` (继承 `QObject`) — AI 上下文中介者
- `AiService` — AI 服务 (静态类)
- `QGraphicsOpacityEffect` — 透明度效果 (面板展开/收起动画)
- `QPropertyAnimation` — 属性动画
- `QNetworkAccessManager` / `QNetworkRequest` / `QNetworkReply` — HTTP 网络请求

- QEventLoop — 同步等待网络响应
- QEvent::MouseButtonPress / Release / MouseMove / ChildAdded — 事件过滤

3.5.1 AiPanelWidget：可折叠的 AI 侧边栏

AI 面板有两种状态：**收起**（只显示 50px 的"AI"按钮条）和**展开**（380px 的完整面板）。

```
AiPanelWidget (QFrame)
├─ m_collapsedStrip (收起状态的窄条)
│   └─ QPushButton "AI" (点击展开)
└─ m_panelContent (展开状态的完整面板)
    ├─ QLabel "AI 建议" (标题)
    ├─ QPushButton "刷新" / "收起" (操作按钮)
    ├─ AiStatusBar (状态指示：本地规则/远程AI/离线)
    ├─ QPushButton "清空上下文"
    └─ AiConversationWidget (对话界面)
        └─ 信号 chatMessageSent → MainWindow 连接到 AiService::chat()
```

展开/收起使用 QGraphicsOpacityEffect + QPropertyAnimation 实现淡入淡出动画。

3.5.2 AiContextMediator：事件过滤器实现"划词提问"

这是项目中最巧妙的设计之一。AiContextMediator 通过 Qt 的 **eventFilter 机制**，在全局范围内拦截鼠标事件，实现"在任意位置选中文本 → 弹出浮窗 → 发送到 AI"的功能。

```

bool AiContextMediator::eventFilter(QObject* watched, QEvent* event) {
    // 1. 子控件添加时，自动让其文本可选
    if (event->type() == QEvent::ChildAdded) {
        makeTextSelectable(qobject_cast<QWidget*>(childEvent->child()));
    }

    // 2. 鼠标按下：记录位置
    if (event->type() == QEvent::MouseButtonPress) {
        m_leftMousePressed = true;
        m_mousePressGlobalPos = mouseEvent->globalPosition().toPoint();
    }

    // 3. 鼠标移动：判断是否拖拽足够距离（区分点击和选中）
    if (event->type() == QEvent::MouseMove && m_leftMousePressed) {
        int distance = (pos - m_mousePressGlobalPos).manhattanLength();
        if (distance >= QApplication::startDragDistance())
            m_draggedEnoughForSelection = true;
    }

    // 4. 鼠标释放：检测选中文本，弹出浮窗菜单
    if (event->type() == QEvent::MouseButtonRelease) {
        QTimer::singleShot(50, this, [this]() { // 延迟50ms等Qt完成选择
            QString selectedText = selectedTextFromWidget(QApplication::focusWidget());
            if (!selectedText.isEmpty() && selectedText.length() > 1)
                showFloatingMenu(QCursor::pos(), selectedText);
        });
    }
}

```

为什么用 eventFilter 而不是继承？ 因为我们需要在**所有**子控件（QLabel、QListWidget、QTableWidget 等）上都能检测选中文本，如果用继承就要改每个控件的基类，而 eventFilter 只需在根 Widget 上安装一次就能拦截所有子控件的事件。

3.5.3 AiService：双模式 AI 引擎

AiService 是一个静态工具类，支持两种工作模式：

```

AiService::chat(data)
|
|— 检测 isAiServerAvailable()
|   |— 使用 QNetworkAccessManager::get() 请求 /health 端点
|   |   QEventLoop 同步等待, QTimer::singleShot 设置 2000ms 超时
|   |   缓存结果 15 秒（可用）或 3 秒（不可用）避免频繁探测
|   |
|   |— 远程模式 (Remote)
|       |— callAiServer() → QNetworkAccessManager::post()
|       携带 JSON body, QJsonDocument::fromJson() 解析响应
|
|— 本地规则模式 (Local)
    |— chatLocal() → generateLocalReply()
        analyzeLocal() →
            buildCourseAdvice() // 基于课程数据生成建议
            buildGoalAdvice()   // 基于目标数据生成建议
            buildExperienceAdvice() // 基于经历数据生成建议
            // ... 每种类型都有独立的规则引擎

```

AI 对话的完整数据流:

用户在 AiConversationWidget 输入消息

→ 信号 chatMessageSent(const QString& message)

→ MainWindow 中的 lambda 槽函数:

```

m_aiPanel->setOutput("AI 正在思考中...")
QApplication::setOverrideCursor(Qt::WaitCursor) // 显示等待光标
QJsonObject result = AiService::chat(payload)   // 同步调用
QApplication::restoreOverrideCursor()           // 恢复光标
m_aiPanel->setOutput(formatted_reply)            // 显示结果

```

【讲解提示】AI 模块是最能体现技术深度的部分。建议分三个层次讲解：(1) **面板设计**：展示收起/展开动画；(2) **划词提问**：在某个页面上选中文字，演示浮窗弹出和发送到 AI 的过程；(3) **双模式引擎**：解释本地规则和远程 AI 的切换逻辑。可以这样说："AiContextMediator 用了 Qt 的事件过滤器模式——这是观察者模式的一种实现。我只在根 Widget 上安装了一个 eventFilter，就能拦截整个窗口内所有控件的鼠标事件，避免了修改每个控件基类的麻烦。"

3.6 数据持久化层

涉及的 Qt 类:

- DaoBase (基类) — 持有 QSqlDatabase 连接
- CourseDao / GoalDao 等 10 个 DAO 类 — 继承 DaoBase
- QSqlDatabase — 数据库连接管理
- QSqlQuery — SQL 查询执行
- QSqlError — 错误处理

DAO 基类设计：

```
class DaoBase {
public:
    DaoBase() : m_db(QSqlDatabase::database()) {
        if (!m_db.isOpen()) {
            Logger::warning("数据库未打开，尝试打开...");
            m_db.open();
        }
    }
protected:
    QSqlDatabase m_db;
};
```

所有 DAO 继承 DaoBase，自动获取数据库连接。QSqlDatabase::database() 返回默认连接（由 main_desktop.cpp 中初始化），实现了**连接池的简化版本**。

【讲解提示】数据层讲解简洁即可："所有 DAO 继承自 DaoBase，通过 Qt 的 QSqlDatabase 管理 SQLite 连接，QSqlQuery 执行 SQL 语句。首次启动时自动运行 schema.sql 创建表结构，用户无需手动配置数据库。"

3.8 系统托盘与全局通知

涉及的 Qt 类：

- QSystemTrayIcon — 系统托盘图标
- QMenu — 托盘右键菜单
- QAction — 菜单动作（打开网页预览、退出）
- ToastNotification (继承 QWidget) — 浮动提示通知
- QApplication::style()->standardIcon() — 使用系统标准图标

```
void MainWindow::setupSystemTray() {
    m_trayIcon = new QSystemTrayIcon(this);
    m_trayIcon->setIcon(QApplication::style()->standardIcon(QStyle::SP_ComputerIcon));
    m_trayMenu = new QMenu(this);
    m_openBrowserAction = m_trayMenu->addAction("打开网页预览");
    m_quitAction = m_trayMenu->addAction("退出");
    m_trayIcon->setContextMenu(m_trayMenu);
    connect(m_trayIcon, &QSystemTrayIcon::activated, this, &MainWindow::onTrayActivated);
    m_trayIcon->show();
}
```

ToastNotification 提供了一个简洁的静态调用方式：

```
ToastNotification::display(this, "对照同学已添加。");
```

【讲解提示】系统托盘功能可以快速演示：最小化应用后在托盘区找到图标，右键展示菜单。这个功能体现了"桌面应用"与"网页应用"的差异——桌面应用能驻留在系统托盘中。>

四、技术难点与解决方案

4.1 MainWindow 从 5000 行重构到 766 行

难点：原始 MainWindow 承载了所有逻辑——页面创建、数据刷新、后端管理、AI 集成、UI 布局，代码超过 5000 行，可维护性极差。

解决方案：职责分离 + 四大协调器

职责	原始位置	重构后	核心 Qt 类
页面切换控制	MainWindow::onNavigationChanged()	AppShellController	QStackedWidget , QLabel
数据刷新协调	各页面手动调用 refresh()	DataRefreshCoordinator	QObject::connect() 信号槽
后端进程管理	MainWindow 内嵌 QProcess	BackendRuntimeController	QProcess , QProgressBar
AI 上下文传递	MainWindow::eventFilter()	AiContextMediator	QEvent , QMouseEvent

另外，页面注册使用 QMap<int, std::function<void()>> 替代 switch-case，新增页面只需一行注册代码。

【讲解提示】重构部分是很好的技术深度展示。可以说："最初我把所有代码都写在 MainWindow 里，很快就发现维护不动了。于是我按照'单一职责原则'，把不同功能拆分到四个独立的协调器中。重构后 MainWindow 只剩 766 行，只负责'组装'，不负责'干活'。" **建议用代码对比的方式展示重构前后的差异。**

4.2 跨页面数据一致性

难点：7 个数据管理页面各自独立操作数据库，修改一个页面的数据后，其他相关页面（如总览页、分析页）需要同步更新。

解决方案：DataDomain 枚举 + 信号槽自动刷新

```

// DataDomain.h - 定义数据域枚举
enum class DataDomain {
    Courses, Roles, Achievements, Experiences,
    Activities, Goals, Jobs, Analysis, All
};

// 每个页面发出 dataChanged 信号
signals:
    void dataChanged(DataDomain domain);

// DataRefreshCoordinator 统一监听
void DataRefreshCoordinator::connectSignals() {
    connect(m_courses, &CoursesPage::dataChanged, this, &DataRefreshCoordinator::refreshByDomain);
    connect(m_goals, &GoalsPage::dataChanged, this, &DataRefreshCoordinator::refreshByDomain);
    // ... 所有页面都连接
}

void DataRefreshCoordinator::refreshByDomain(DataDomain domain) {
    // 总是刷新总览页和时间轴页
    m_overview->refresh();
    m_timeline->refresh();

    // 根据域刷新相关页面
    switch (domain) {
        case DataDomain::Goals:
            m_goals->refresh();
            m_analysis->refresh();
            m_resume->refresh();
            break;
        // ...
    }
}

```

【讲解提示】数据一致性可以用一个具体场景讲解："假设用户在目标页添加了一个新目标，这时总览页的目标进度、分析页的建议、简历页的素材推荐都需要更新。DataRefreshCoordinator 通过 Qt 的信号槽机制，自动完成这个联动——用户完全无感知。"

4.3 AI 双模式引擎与优雅降级

难点：远程 AI 服务可能不可用（未启动、网络问题），如何保证系统始终能提供 AI 功能？

解决方案：三层降级策略

```

AiService::chat(data)
|
|— 第1层：检查远程 AI 可用性（缓存15秒）
|   isAiServerAvailable() → QNetworkAccessManager::get("/health")
|   QEventLoop + QTimer::singleShot(2000ms) 超时保护
|
|— 第2层：远程 AI 可用 → callAiServer()
|   QNetworkAccessManager::post() 发送 JSON 请求
|   QJsonDocument 解析响应
|
|— 第3层：远程不可用 → 本地规则引擎
|   chatLocal() → generateLocalReply()
|   analyzeLocal() → buildCourseAdvice() / buildGoalAdvice() / ...

```

AiStatusBar 实时显示当前模式（本地规则 / 远程 AI / 连接中），用户始终知道 AI 的工作状态。

【讲解提示】这部分可以现场演示：(1) 启动远程 AI 服务，展示"远程 AI 模式"；(2) 关闭远程服务，展示自动切换到"本地规则模式"；(3) 观察 AiStatusBar 的状态变化。可以说："这种'优雅降级'的设计思想来自实际工程经验——系统不应该因为一个外部服务不可用就完全瘫痪。"

4.4 事件过滤器实现全局文本选中

难点：需要在所有类型的控件（QLabel、QListWidget、QTableWidget、QTextEdit 等）上都能检测用户选中的文本，但这些控件的基类各不相同。

解决方案：eventFilter 模式 + 递归 makeTextSelectable

```

void AiContextMediator::bindRootWidget(QWidget* root) {
    m_rootWidget = root;
    m_rootWidget->installEventFilter(this); // 在根 Widget 安装事件过滤器
    makeTextSelectable(m_rootWidget);      // 递归设置子控件文本可选
}

bool AiContextMediator::eventFilter(QObject* watched, QEvent* event) {
    // ChildAdded 事件：新添加的子控件也自动设为可选
    if (event->type() == QEvent::ChildAdded) {
        makeTextSelectable(qobject_cast<QWidget*>(childEvent->child()));
    }
    // MouseMove + MouseButtonRelease: 检测拖拽选中 → 提取文本 → 弹出浮窗
    // ...
}

```

为什么用 manhattanLength() 判断距离？ 因为曼哈顿距离计算只需要加法和绝对值，比欧氏距离（需要开方）快得多，足以判断"用户是否真的在拖拽选中文字"。

【讲解提示】eventFilter 是 Qt 中一个高级但非常实用的特性。可以这样解释："普通的信号槽只能连接到特定对象的特定信号，但 eventFilter 可以拦截任何 QObject 的任何事件。我在根 Widget 上安装了过滤器，就像在整个窗口上铺了一层'透明薄膜'，所有鼠标事件都会先经过我的处理。"

五、项目亮点总结

5.1 架构设计亮点

设计模式	应用场景	核心类
协调器模式	跨页面数据刷新	DataRefreshCoordinator
策略模式	通用 CRUD 框架	CrudPageController + std::function
中介者模式	AI 上下文传递	AiContextMediator
事件过滤器	全局文本选中检测	AiContextMediator::eventFilter()
模板方法	页面基类	BasePage::refresh() 纯虚函数
观察者模式	数据变更通知	Qt 信号槽 (signals/slots)

5.2 Qt 技术深度

- 属性动画系统：QPropertyAnimation + QParallelAnimationGroup + QEasingCurve 实现流畅 UI 动画
- 事件系统：重写 eventFilter() 实现全局事件拦截
- 信号槽机制：跨组件、跨层的松耦合通信
- 数据库集成：QSqlDatabase + QSqlQuery + QSqlError 完整的 SQLite 操作
- 网络编程：QNetworkAccessManager 实现 HTTP 客户端（AI 服务调用）
- JSON 处理：QJsonDocument / QJsonObject / QJsonArray 全链路 JSON 序列化
- 系统集成：QSystemTrayIcon 系统托盘、QClipboard 剪贴板、QDesktopServices 系统服务
- 状态管理：QSettings 持久化窗口几何信息和用户配置
- 防抖优化：QTimer::setSingleShot() 实现输入防抖

5.3 工程实践亮点

- MainWindow 从 5000+ 行重构到 766 行，体现了代码质量意识
- 6 层架构设计，职责分离清晰
- 151 个文件、15,200 行代码的完整工程
- 完善的文档体系（架构文档、组件文档、开发指南）
- 一键构建脚本 (build_desktop.bat)

【讲解提示】亮点总结部分要自信、简洁。可以用一句话概括："这个项目综合运用了 Qt 的 UI 框架、事件系统、数据库、网络编程、动画系统等多个子系统，不是简单的 CRUD 应用，而是一个有架构设计、有技术深度的完整工程。"

六、总结与展望

6.1 项目总结

通过本次项目开发，我深入实践了以下技术能力：

1. **Qt 6 全栈开发**：从 UI 到数据库到网络，覆盖了 Qt 的核心模块
2. **C++17 现代特性**：std::function、constexpr、结构化绑定、lambda 表达式
3. **分层架构设计**：6 层架构，151 个文件的大型项目管理
4. **设计模式应用**：协调器、策略、中介者、观察者等多种模式的实际运用
5. **重构与优化**：从 5000+ 行到 766 行的代码重构经验

6.2 未来展望

如果有机会继续完善，我计划：

1. **Qt Quick/QML**：将部分 UI 迁移到 QML，实现更丰富的动画效果
2. **多线程优化**：使用 QThread / QtConcurrent 将 AI 调用放到后台线程
3. **云端同步**：通过 QNetworkAccessManager 实现数据云端同步
4. **插件化架构**：支持用户自定义分析规则和 AI 模型

【讲解提示】展望部分体现你的学习热情和技术视野。提到 QML、多线程等方向，说明你不满足于现状，有持续学习的意愿。

致谢

感谢各位老师和同学的聆听！

感谢老师在课程中传授的 Qt 和 C++ 知识，这是完成这个项目的基礎。

感谢同学们在开发过程中提供的宝贵建议。

如果大家有任何问题，欢迎交流讨论。

谢谢大家！

附录：Qt 核心类速查表

Qt 模块	核心类	本项目应用
Core	QObject , QTimer , QSettings , QJsonObject	信号槽基础、防抖、持久化、JSON
Widgets	QMainWindow , QWidget , QStackedWidget , QDialog	主窗口、页面容器、对话框
Layout	QHBoxLayout , QVBoxLayout , QGridLayout , QSplitter	所有页面布局
Controls	QLabel , QPushButton , QLineEdit , QComboBox , QListWidget , QTableWidget	表单和列表
Animation	QPropertyAnimation , QParallelAnimationGroup , QEasingCurve , QGraphicsOpacityEffect	侧边栏折叠、AI面板动画
Sql	QSqlDatabase , QSqlQuery , QSqlError	SQLite 数据库操作
Network	QNetworkAccessManager , QNetworkRequest , QNetworkReply	AI 服务通信
Gui	QClipboard , QDesktopServices , QFileDialog , QSystemTrayIcon	系统集成功能

附录：类协作关系图



附录 A：答辩追问应对策略（备战手册）

重要提示：以下内容不是演讲的一部分，而是答辩前的备战材料。老师在听完汇报后一定会通过追问来验证你对技术的理解深度。请务必熟记以下每个问题的应对思路。

一、致命技术瑕疵追问

Q1: UI 线程阻塞问题（极大概率被问）

老师可能这样问：

"你提到在 `AiService` 中用 `QEventLoop + QTimer::singleShot` 同步等待网络响应，如果网络不好，UI 会不会卡死？Qt 是事件驱动的，这样做是不是违背了 GUI 编程原则？"

标准回答：

"老师您指出的非常对，这确实是目前项目的一个技术妥协。"

在 `AiService::isAiServerAvailable()` 中，我使用了 `QEventLoop` 同步等待 `QNetworkAccessManager` 的响应，并设置了 2000ms 的超时保护：

```
QEventLoop loop;
QNetworkReply* reply = manager.get(request);
QTimer::singleShot(kAiHealthTimeoutMs, &loop, &QEventLoop::quit);
QObject::connect(reply, &QNetworkReply::finished, &loop, &QEventLoop::quit);
loop.exec(); // 阻塞主线程，但有超时保护
```

当时为了快速实现 AI 双模式的**同步降级逻辑**（先探测远程是否可用，不可用则立即切换到本地），采用了这种同步写法。超时 2 秒是可接受的折中——用户只会感觉到一次短暂的'思考'。

但如果要优化到生产级别，我会做以下改造：

1. 将 `AiService` 的网络调用改为**纯异步模式**——`QNetworkAccessManager::get()` 本身就是异步的，只需连接其 `finished` 信号到一个回调槽函数
2. 在等待期间显示 `QProgressBar` 或加载动画，而不是冻结 UI
3. 或者使用 `QtConcurrent::run()` 将整个 AI 调用放入 `QThreadPool`，通过信号槽将结果传回主线程更新 UI"

追问升级：如果老师继续问"那你为什么不一开始就用异步？"

"因为异步模式会让代码复杂度显著增加——需要处理回调嵌套、生命周期管理（如果用户在 AI 响应回来之前关闭了页面怎么办）、以及错误重试的状态机。在课程项目的时间约束下，我选择了'先跑通再优化'的策略。这也是软件工程中'迭代开发'思想的体现。"

Q2: C++ 内存管理（必问）

老师可能这样问：

"C++ 最怕内存泄漏，你这 1.5 万行代码里 `new` 了这么多对象（动画、对话框、Widget），你是怎么管理内存的？有没有用智能指针？"

标准回答：

"本项目主要依赖 **Qt 的对象树（Object Tree）** 机制来管理内存。

所有继承自 `QObject` 的类——包括 `Widget`、`Controller`、`Animation`——在创建时都传入了 `this` 作为 `parent` 参数：

```
m_sidebar = new SidebarWidget(this);           // parent = MainWindow
m_shellController = new AppShellController(this); // parent = MainWindow
anim1->setParent(this);                         // 动画对象挂到 Widget 下
```

当 `MainWindow` 在析构函数中销毁时，Qt 会**递归销毁**其所有子对象，这是 `QObject` 的内置机制，不需要手动 `delete`。

对于**非 `QObject` 的临时对象**（如 `QJsonObject`、`QJsonDocument`），它们是栈上对象或值类型，作用域结束自动销毁。

对于**需要手动管理的特殊对象**，我严格遵守了 Qt 的规范：

- `QNetworkReply` 使用后立即调用 `reply->deleteLater()`（在 `AiService` 中有 3 处）
- `QParallelAnimationGroup` 动画完成后通过
`connect(group, &QParallelAnimationGroup::finished, group, &QObject::deleteLater)` 自动释放
- `ToastNotification` 淡出动画结束后自动调用 `deleteLater()` 销毁自身
- 浮窗菜单 `m_floatingWidget` 在隐藏时调用 `deleteLater()`

项目中**没有使用 `std::unique_ptr` 或 `std::shared_ptr`**，因为 Qt 的对象树机制已经足够，混用智能指针反而可能导致'双重释放'问题——这是 Qt 官方文档明确建议的。"

加分点：如果老师问到 `deleteLater()` 和 `delete` 的区别：

"`deleteLater()` 不是立即删除，而是向事件循环发送一个 `DeferredDelete` 事件，等当前事件处理完毕后再删除。这避免了在信号槽调用链中删除正在使用的对象导致的崩溃。比如 `QNetworkReply` 的 `finished` 信号回调中，`reply` 对象可能还在被 Qt 内部使用，所以必须用 `deleteLater()` 而不是直接 `delete`。"

二、AI 使用与原创性验证（核心必答题）

特别说明：这是答辩中**最敏感、最可能被追问**的话题。老师问这个问题的目的不是要"抓你"，而是想看你的**诚实态度**和**技术判断力**。以下回答策略的核心思路是：**坦诚承认 AI 辅助 + 用具体例子划清边界 + 强调架构决策是自己的**。

Q3-A：你用了 AI 吗？用了多少？

老师可能这样问：

"15000 行代码、151 个文件，一个人一学期是怎么写完的？你用了 AI 代码生成工具吗？"

标准回答：

"老师，我确实在开发过程中使用了 AI 工具。但我对 AI 的使用有明确的边界和方法论——我把它称为**原型驱动 + AI 辅助生成**的开发模式。

具体来说：

- 1. 我**亲自设计并手写**了一个标准的模板类（比如 CourseDao ），确立命名规范、内存管理方式和核心接口
- 2. 然后利用 AI 工具，输入不同数据表的字段定义，让 AI 基于我的模板**批量生成**其他同质化的类（如 GoalDao 、 RoleDao 等）
- 3. 最后我对 AI 生成的代码进行 **Code Review（代码审查）** 和微调

这种做法让我从机械的重复劳动中解放出来，将核心精力聚焦于**系统架构设计、协调器模式的应用以及复杂业务逻辑的实现上。**"

【讲解提示】**千万不要说"我让 AI 帮我复制改写"**——这会让老师觉得你在偷懒。用"原型驱动 + AI 辅助生成"这个说法，体现了你有方法论、有流程管控。老师听到"Code Review"和"原型模板"这些词，会认为你具备工程化思维。

话术升级对照表：

危险说法（不要说）	专业说法（建议说）
"Model 层差不多，让 AI 复制改写"	"我采用原型驱动 + AI 辅助生成的开发模式"
"为了省事让 AI 写的"	"为了提高开发效率并保证代码规范"
"AI 写完我没怎么改"	"AI 生成后我负责 Code Review 和微调"
"这些代码没什么技术含量"	"这些是结构相似的样板代码（Boilerplate Code）"

理论支撑——如果老师追问"为什么这么做"，可以用以下三点回应：

- 1. **符合 DRY 原则（Don't Repeat Yourself）**：人类开发者不应该把时间浪费在机械地敲击重复的 CRUD 代码上。利用 AI 生成样板代码，是现代开发中消除重复劳动的最佳实践。
- 2. **关注点分离（Separation of Concerns）**：我的核心竞争力在于架构设计（六层架构、协调器、事件过滤器），而不是"能多快写完 10 个 DAO 类的 SQL 语句"。让 AI 处理底层重复代码，我专注于高层架构——这是**架构师的思维**。
- 3. **现代开发范式（AI-Assisted Development）**：在工业界，使用 GitHub Copilot 或大模型生成 DTO、DAO 和单元测试已经是标准操作。我在这个项目中提前实践了这种**人机协同的开发流程**。

Q3-B：AI 具体帮你写了什么？（举具体例子）

老师可能这样问：

"能具体说说哪些代码是 AI 帮你写的吗？"

标准回答——用“三类代码”来分类说明：

"我把代码分成三类，AI 的参与程度完全不同："

第一类：纯样板代码（AI 辅助率 ~80%，但价值低）

"这类代码模式高度重复，AI 能快速生成，但我需要逐行审查和调整。"

例子 1：DAO 层的 SQL 操作代码

项目有 10 个 DAO 类（CourseDao、GoalDao、RoleDao 等），每个都有 getAll()、getById()、create()、update()、remove() 五个方法，结构几乎一样——只是表名和字段不同。比如 CourseDao::create()：

```
bool create(const Course& course) {
    QSqlQuery query(m_db);
    query.prepare("INSERT INTO courses (name, code, credits, ...) "
                  "VALUES (:name, :code, :credits, ...)");
    query.bindValue(":name", course.name);
    query.bindValue(":code", course.code);
    query.bindValue(":credits", course.credits);
    // ... 12 个字段的绑定
    return query.exec();
}
```

这种代码 AI 可以根据 Course 模型类快速生成，但**字段名、绑定顺序、SQL 语法**我都要逐一检查。

例子 2：UI 布局代码

比如 CoursesPage::setupUi() 中大量的 new QLabel、addWidget、setContentsMargins、setObjectName 调用，这类代码没有逻辑，纯粹是“体力活”，AI 生成效率很高。

第二类：业务逻辑代码（AI 辅助率 ~40%，需要深度修改）

"这类代码有明确的业务逻辑，AI 能提供初始框架，但我需要大量修改。"

例子 1：CourseService::getStatistics() 的 GPA 计算逻辑

AI 生成了基本的遍历和求和代码，但以下逻辑是我自己设计的：

- GPA 的两种计算标准（standard 4.0 制 vs 4.33 制）的选择逻辑
- 学分进度的百分比计算和上限封顶（std::min(..., 100.0)）
- 按学期分组统计的 QMap<QString, QList<Course>> 数据结构选择

例子 2：AiService 的双模式切换逻辑

AI 帮我写了 QNetworkAccessManager 的基本调用代码，但**双模式切换的核心设计**是我自己的：

- 健康检查的缓存策略（可用时缓存 15 秒，不可用时 3 秒后重试）
- 本地规则引擎的 6 种建议类型（课程/目标/经历/简历/成就/综合）
- 优雅降级的三层策略（远程可用 → 远程调用 → 本地规则）

第三类：架构设计代码（AI 辅助率 ~0%，完全自主设计）

"这类代码是系统的'骨架'，AI 无法替代——因为它需要对整个系统有全局理解。"

例子 1：CrudPageController 的策略模式设计

这个类的核心设计决策——用 `std::function` 回调注入不同页面的具体逻辑——是我自己想出来的。AI 不知道我有 7 个 CRUD 页面需要复用同一套增删改查流程，也不知道我需要用 `JsonObject` 作为中间数据格式来统一不同数据类型。

例子 2：AiContextMediator 的 eventFilter 设计

我需要在所有类型的控件上实现"划词提问"功能。这个需求的解决方案——用 `eventFilter` 拦截全局鼠标事件、用 `qobject_cast` 逐类型检测选中文本、用 `manhattanLength` 判断拖拽距离——是我自己设计的。AI 可以帮我写 `eventFilter` 的框架代码，但它不知道为什么要用 `manhattanLength` 而不是欧氏距离，也不知道为什么要处理 `QEvent::ChildAdded` 来支持动态添加的控件。

例子 3：DataRefreshCoordinator 的信号槽拓扑

12 个页面的数据刷新联动，如果两两连接信号槽会产生 132 条连接线。我设计了星型拓扑——所有页面只发出 `dataChanged(DataDomain)` 信号，协调器集中维护刷新规则。这个**架构决策**需要对整个系统的数据流有全局理解，AI 无法替代。

例子 4：从 5000 行重构到 766 行

重构的核心工作是**识别职责边界**——哪些代码属于"壳层控制"（拆到 `AppShellController`）、哪些属于"数据刷新"（拆到 `DataRefreshCoordinator`）、哪些属于"后端管理"（拆到 `BackendRuntimeController`）。这需要理解每段代码的**依赖关系**和**变化原因**，是纯粹的架构思考，不是代码生成能解决的。

Q3-C：你怎么保证 AI 生成的代码是正确的？

老师可能这样问：

"AI 生成的代码你怎么保证没有 Bug？你做了什么验证？"

标准回答：

"我对 AI 生成的代码有三层验证：

1. **编译验证**：Qt 的 C++ 编译器会检查类型安全、未声明变量、内存访问等问题。AI 生成的代码如果编译不过，我会逐个修复编译错误。
2. **运行时验证**：每个功能完成后我都会手动测试。比如 DAO 层的 SQL 语句，我会通过 UI 添加一条数据，然后用 SQLite 客户端工具直接查看数据库文件 `pdp.db`，确认数据是否正确写入。

3. **代码审查**：我会逐行阅读 AI 生成的代码，理解每行的作用。不理解的代码绝不提交——这是我的原则。比如 AI 生成的 `QNetworkReply` 代码如果没有调用 `deleteLater()`，我会手动补上，因为我知道不释放会导致内存泄漏。

另外，项目中 `docs/` 目录下的 `REGRESSION_CHECKLIST.md` 文件记录了回归测试清单，每次重构后我都会按照清单逐一验证各页面的功能。"

Q3-D：如果老师问"那你能不能现场手写一段核心代码？"

防御话术：

"当然可以。我可以现场写出以下任何一个核心模块的关键代码：

1. `DataRefreshCoordinator::connectSignals()` 的信号槽连接逻辑
2. `AiContextMediator::eventFilter()` 的鼠标事件检测流程
3. `CrudPageController::executeEdit()` 的通用编辑流程
4. `AiService::isAiServerAvailable()` 的健康检查逻辑

因为这些代码的**设计思路**是我自己想出来的，我能清晰地解释每一行代码的意图和它在系统中的位置。"

Q3-E：如何看待 AI 辅助编程？

老师可能这样问：

"你觉得用 AI 写作业算作弊吗？"

标准回答：

"我认为关键在于**用 AI 做什么**。

用 AI 生成样板代码、查询 API 用法、调试编译错误——这和用搜索引擎查 Stack Overflow 没有本质区别，是正常的工程实践。在实际工作中，开发者使用 AI 辅助编程已经是行业趋势。

但如果用 AI 直接生成整个系统的架构设计，然后不理解就提交——这就失去了学习的意义。

我的做法是：**AI 帮我加速了'从想法到代码'的过程，但'从需求到想法'的思考过程完全是我自己的**。比如用 `eventFilter` 解决全局文本选中'这个设计方案，我需要先理解 Qt 的事件系统、比较继承和事件过滤的优劣、考虑动态添加控件的边界情况——这些思考是 AI 无法替代的。

通过这个项目，我不仅学会了 Qt 和 C++，更重要的是学会了**架构设计的思维方式**——如何拆分职责、如何解耦模块、如何在复杂度和可维护性之间做权衡。这些能力是无法通过 AI 生成获得的。"

Q3-F：AI 生成的代码有 Bug 你怎么发现？

老师可能这样问：

"AI 生成的代码如果有一处隐蔽的 Bug——比如字段类型映射错了，或者内存泄漏——你怎么发现？"

标准回答：

"老师，这也是我坚持进行 Code Review 的原因。AI 只是生成器，我是最终的把关人。

第一道防线：编译期检查。C++ 是强类型语言，如果 AI 把 `int` 错写成 `QString`，或者 Qt 的信号槽签名不匹配，编译器（MSVC/GCC）会直接报错，无法通过编译。这是静态类型语言相比 Python 等动态语言的优势。

第二道防线：运行时测试。我在 DAO 层每次修改后都会手动测试——通过 UI 添加一条数据，然后用 SQLite 客户端工具直接查看 `pdp.db` 文件，确认字段映射和数据类型是否正确。

第三道防线：规范约束。因为我给 AI 的 Prompt 中包含了严格的编码规范（比如必须使用 `Q_PROPERTY`、必须继承 `QObject`、`QNetworkReply` 必须调用 `deleteLater()`），这大大降低了 AI 产生低级错误的概率。

举一个实际的例子：AI 生成的 `AIService` 代码最初没有在 `QNetworkReply` 使用后调用 `deleteLater()`，我在 Code Review 时发现了这个问题并手动补上——因为我理解 Qt 的内存管理机制。"

Q3-G：为什么不用 ORM 或万能基类来避免重复代码？

老师可能这样问：

"既然 Model/DAO 层这么重复，你为什么不用 ORM 框架，或者写一个万能基类通过反射来处理？"

标准回答：

"老师，我确实评估过这两种方案，但最终选择了'独立类 + AI 辅助生成'，原因有两点：

第一，C++ 缺乏原生反射机制。不像 Java 或 C# 有强大的反射，C++ 要实现通用的 ORM 往往需要引入宏定义或复杂的模板元编程，这会大大增加编译时间和代码复杂度。Qt 的元对象系统（`Q_OBJECT`）虽然提供了部分反射能力，但它的设计初衷是支持信号槽，不是用来做数据库映射。

第二，性能与可控性。这个系统涉及多表查询和复杂统计——比如 `CourseService::getStatistics()` 中的按学期分组、GPA 加权计算、学分进度百分比等。手写具体的 SQL 语句虽然代码量大，但性能最高、SQL 调优最灵活。

所以我选择了**扁平化的独立类 + AI 辅助生成**的方案，这是在开发效率、运行性能和代码可控性之间的最佳权衡。"

Q3-H：如果现在要加一个新数据表，你需要多久？

老师可能这样问：

"如果我现在要求你加一个新的数据表，比如'实习记录表'，你需要多久能完成 Model + DAO + Service？"

标准回答：

"大概 3 到 5 分钟就可以完成核心代码。

因为我已经沉淀了一套标准的模板。流程是这样的：

1. 在 `schema.sql` 中添加 `CREATE TABLE internships (...)` 语句
2. 参照 `Course.h` 的结构, 用 AI 生成 `Internship` 模型类 (字段、`toDict()`、`fromDict()`)
3. 参照 `CourseDao.h` 的结构, 用 AI 生成 `InternshipDao` (CRUD 方法)
4. 参照 `CourseService.h` 的结构, 生成 `InternshipService`
5. 创建 `InternshipsPage` 和 `InternshipEditorDialog`
6. 在 `MainWindow` 中注册页面, 在 `DataRefreshCoordinator` 中添加刷新逻辑

前 4 步用 AI 生成, 后 2 步我手动完成——因为涉及架构层面的集成。这证明了我设计的**四层架构具备高可扩展性**——新增一个业务实体不需要修改任何已有代码, 只需要插入新的层。"

完美逻辑闭环：将 AI 使用与架构重构串联

【讲解提示】如果老师对你的 AI 使用表示认可, 可以主动把话题引导到架构重构上, 形成一个完整的叙事:

"老师, 总结来说, 15000 行代码中确实有相当一部分是 Model 层、DAO 层和 UI 布局的样板代码, 这部分我利用了 AI 辅助生成, 保证了开发速度。

但系统核心的 `MainWindow`、四大协调器、`AiContextMediator` 的事件拦截器、`CrudPageController` 的策略模式——这些是我经过深思熟虑、从 5000 行臃肿代码一步步重构、解耦到 766 行的精华。

AI 帮我干脏活累活, 我专注于架构设计和核心逻辑重构——这就是我能在一学期内独立完成这个项目的秘诀。"

这样说, 老师不仅不会质疑原创性, 反而会觉得你具备**工程素养、架构能力和现代工具使用能力**。

三、架构与设计模式深度追问

Q4: SQLite 并发与锁问题

老师可能这样问:

"你用了 SQLite, 如果 HTTP Server 收到并发请求, 同时 UI 也在读写数据库, 会不会报 `database is locked` ?"

标准回答:

"这是一个很好的问题。目前系统主要是**单用户桌面端场景**, 并发量极低——UI 线程操作数据库时, HTTP Server 的请求也会排队处理。

我在 `schema.sql` 中设置了 `PRAGMA foreign_keys = ON` 来保证数据完整性。SQLite 默认使用**文件锁**模式, 写操作会阻塞其他读写。

如果要支持更高的并发, 我了解到可以做以下优化:

1. 开启 **WAL (Write-Ahead Logging) 模式**: `PRAGMA journal_mode=WAL;`, 这样读操作不会被写操作阻塞

2. 将所有数据库写操作放入**事务**中（`BEGIN TRANSACTION ... COMMIT`），减少锁的持有时间
3. 为 HTTP Server 的每个请求创建**独立的数据库连接**（Qt 的 `QSqlDatabase::addDatabase()` 支持命名连接）

但在当前的单用户桌面场景下，SQLite 的默认行为已经完全满足需求。"

注意：如实回答——项目中确实没有设置 WAL 模式，但你知道这个知识点。

Q5：协调器模式与中介者模式的区别

老师可能这样问：

"你提到了 `DataRefreshCoordinator`（协调器）和 `AiContextMediator`（中介者），这两个模式有什么区别？为什么不直接用信号槽互相连接？"

标准回答：

"这两个模式解决的是**不同层面的解耦问题**。

直接用信号槽的问题：如果 12 个页面两两连接信号槽，会产生 $N \times (N-1)$ 条连接线，形成'蜘蛛网'式的依赖关系。每新增一个页面，就要和所有已有页面建立连接。

`DataRefreshCoordinator`（协调器）解决的是数据刷新的一对多广播问题。它是一个**星型拓扑的中心节点**——所有页面只负责发出 `dataChanged(DataDomain)` 信号，不关心谁需要刷新；协调器集中维护'哪个域变化了应该刷新哪些页面'的映射规则。这符合**迪米特法则（最少知识原则）**。

`AiContextMediator`（中介者）解决的是不同类控件与 AI 面板之间的通信问题。它通过 `eventFilter` 拦截全局鼠标事件，检测文本选中，然后转发给 `AiPanelWidget`。如果没有这个中介者，每个需要支持'划词提问'的控件（`QLabel`、`QListWidget`、`QTableWidget` 等）都要自己实现选中检测和 AI 通信逻辑。

简单来说：

- 协调器 = **消息路由器**（'数据变了' → '谁需要知道'）
- 中介者 = **协议转换器**（'鼠标事件' → 'AI 上下文'）"

Q6：为什么要用 `eventFilter` 而不是继承？

老师可能这样问：

"你想让所有控件都支持文本选中，为什么不写一个 `SelectableWidget` 基类让所有控件继承？"

标准回答：

"因为项目中的控件**基类各不相同**——有的继承 `QLabel`，有的继承 `QListWidget`，有的继承 `QTableWidget`，它们的共同祖先只有 `QWidget`。如果用继承方案，要么写多个 `SelectableLabel`、`SelectableListWidget`、`SelectableTableWidget`，要么用**多重继承**（这在 Qt 中会破坏 `Q_OBJECT` 宏的元对象系统）。

而 `eventFilter` 只需要在**根 Widget 上安装一次**，就能拦截其所有子控件的事件，包括未来动态添加的控件。代码量从'每个控件类改一遍'变成了'一处代码解决所有问题'。

而且我还处理了 `QEvent::ChildAdded` 事件——当有新的子控件被动态添加时，自动调用 `makeTextSelectable()` 让它也支持文本选中。这在继承方案中很难做到。"

四、Qt 技术细节追问

Q7：信号槽的连接类型

老师可能这样问：

"你代码里的 `connect` 都是默认的 `AutoConnection`，如果信号和槽在不同线程怎么办？"

标准回答：

"目前项目中所有信号和槽都在**主线程（UI 线程）**中，所以使用默认的 `Qt::AutoConnection` 就等同于 `Qt::DirectConnection` ——信号发射时直接调用槽函数。

如果未来将 AI 调用放到后台线程，信号和槽就会跨线程，此时 `AutoConnection` 会自动变为 `Qt::QueuedConnection` ——信号发射时将事件投递到接收者所在线程的事件队列中，由目标线程的事件循环来调用槽函数。这保证了**线程安全**——UI 更新永远在主线程执行。

我了解 Qt 还支持 `Qt::BlockingQueuedConnection`（阻塞等待目标线程处理完毕）和 `Qt::UniqueConnection`（避免重复连接），但在当前项目中没有使用场景。"

Q8：Q_OBJECT 宏的作用

老师可能这样问：

"你的每个头文件都有 `Q_OBJECT` 宏，它到底做了什么？不写会怎样？"

标准回答：

"`Q_OBJECT` 宏展开后会做三件事：

1. 声明 `staticMetaObject` 静态成员——这是 Qt **元对象系统**的核心数据结构
2. 声明 `qt_metacall()` 虚函数——信号槽机制通过它来调用槽函数

3. 声明 `tr()` 和 `metaObject()` 等方法

如果不写 `Q_OBJECT`：

- 信号槽仍然可以编译通过（因为 `connect` 是模板函数），但运行时不会正确调用槽函数——因为缺少元对象信息，Qt 无法通过字符串匹配找到槽函数
- 无法使用 `qobject_cast` 进行安全的类型转换
- 无法使用 Qt 属性系统（`Q_PROPERTY`）

所以我的项目中，所有需要使用信号槽的类（Widget、Controller、Mediator 等）都加上了 `Q_OBJECT` 宏。”

Q9：QSettings 存了什么？数据存在哪里？

老师可能这样问：

“你用了 `QSettings` 做持久化，它和 `SQLite` 有什么区别？存了什么数据？”

标准回答：

“`QSettings` 是 Qt 提供的轻量级键值对存储，适合保存少量配置信息。在 Windows 上它底层使用注册表，在 macOS 上使用 **plist 文件**，在 Linux 上使用 **INI 文件**。

本项目中 `QSettings` 存储的数据包括：

- **窗口几何信息**：`settings.value("geometry")` 和 `settings.setValue("geometry", saveGeometry())` ——记住窗口大小和位置
- **学生个人信息**：姓名、学号、头像路径等——通过 `StudentInfoCard` 读写
- **简历配置**：头像路径 `settings.value("profile/avatar")`

而 `SQLite` 存储的是结构化业务数据——课程、目标、经历等大量记录。

简单来说：`QSettings` 存‘配置’，`SQLite` 存‘数据’。”

五、工程实践追问

Q10：CMake 构建系统是怎么配置的？

老师可能这样问：

“你的项目用了 CMake，能说说 `CMakeLists.txt` 里是怎么配置 Qt 模块的吗？”

标准回答：

“在 `CMakeLists.txt` 中，我使用了 Qt 6 的 `find_package` 来查找所需的模块：

```
find_package(Qt6 REQUIRED COMPONENTS Core Widgets Sql Network)
```

然后用 `qt_add_executable` 创建可执行目标，并通过 `target_link_libraries` 链接这些模块。Qt 6 的 CMake 集成会自动处理 **MOC（元对象编译器）**——当头文件中有 `Q_OBJECT` 宏时，CMake 会自动调用 `moc` 生成元对象代码，不需要手动运行。

项目还提供了 `build_desktop.bat` 一键构建脚本，封装了 `cmake -B build && cmake --build build` 的完整流程。"

Q11：错误处理是怎么做的？

老师可能这样问：

"你的代码里有没有做错误处理？比如数据库写入失败、网络请求超时怎么办？"

标准回答：

"我在多个层面做了错误处理：

数据库层：

- `DaoBase` 构造时检查 `m_db.isOpen()`，如果未打开则尝试重新打开
- `main_desktop.cpp` 中每个 SQL 语句执行后都检查 `query.exec()` 返回值，失败时记录 `Logger::warning`
- 数据库初始化失败时弹出 `QMessageBox::critical` 并退出程序

网络层：

- `AiService` 的 `QNetworkReply` 检查 `reply->error()` 是否为 `QNetworkReply::NoError`
- 设置了 `request.setTransferTimeout()` 超时保护
- 健康检查有 2000ms 超时，AI 生成有 180000ms 超时
- 网络失败时自动降级到本地规则引擎

UI 层：

- 删除操作前弹出 `QMessageBox::question` 确认对话框
- 对话框有 `validateAndAccept()` 校验逻辑，输入不合法时阻止提交
- 操作完成后通过 `ToastNotification::display()` 给用户反馈

日志系统：

- 自定义 `Logger` 类，支持 `info`、`warning`、`error` 三个级别
- 关键操作都有日志记录，便于排查问题"

六、"压力测试"级追问

Q12：如果让你重新做这个项目，你会怎么改？

老师可能这样问：

"你觉得这个项目最大的不足是什么？如果重做会怎么改？"

标准回答：

"最大的不足有三点：

1. **主线程同步网络调用**：前面提到的 `QEventLoop` 阻塞问题。重做时我会从一开始就设计**异步架构**——所有网络操作通过信号槽异步回调，UI 层只负责发送请求和接收结果。
2. **没有单元测试**：项目有 `smoke_test.cpp` 和 `crud_regression.py`，但覆盖面不够。重做时我会为 Service 层和 DAO 层编写完整的单元测试——Qt Test 框架支持数据驱动测试，可以高效覆盖各种边界情况。
3. **没有引入 QML**：当前 UI 全部用 C++ Widgets 代码实现，动画效果有限。如果重做，我会用 **Qt Quick/QML** 实现前端，C++ 只负责后端逻辑——QML 的声明式语法做动画和响应式布局远比 Widgets 强大。

但这些'不足'也是项目迭代的方向——先用 Widgets 跑通核心功能，再逐步迁移到 QML，这是'增量开发'的思路。"

Q13：你从这个项目中学到了什么？

老师可能这样问：

"如果用一句话总结，你从这个项目中最大的收获是什么？"

标准回答：

"最大的收获是理解了**架构先行**的重要性。

项目初期我把所有代码写在 `MainWindow` 里，5000 行之后完全无法维护。这次经历让我深刻理解了为什么要分层、为什么要解耦——不是为了'显得专业'，而是为了让代码**能改、能扩展、能调试**。

从 5000 行重构到 766 行的过程，让我真正理解了'单一职责原则'和'依赖倒置原则'——这些设计原则在课本上读过，但只有在真实项目中踩过坑，才能内化为自己的工程直觉。"

七、答辩时间分配建议

环节	时长	内容
演讲	10-12 min	背景(1min) + 架构与重构(3min) + 核心功能演示(4min) + 难点攻克(3min)
演示	2-3 min	侧边栏动画 → 课程CRUD → 分析页 → 简历导出 → AI划词提问
答辩	3-5 min	回答 2-3 个追问

八、现场演示防翻车预案

- 1. **录制备用视频**：提前录制 2 分钟的高清演示视频嵌入 PPT，如果现场 Demo 失败立刻切换
- 2. **预热 AI 服务**：答辩前 5 分钟启动 AI Server，确保模型加载完成
- 3. **准备测试数据**：用 `seed_db.py` 预填充数据库，避免演示时手动输入浪费时间
- 4. **关闭网络依赖**：如果网络环境不确定，提前切换到本地规则引擎模式并告知老师
- 5. **准备白板/纸笔**：如果需要解释架构或手写代码，随时可以画图

（以上为答辩备战材料，祝答辩顺利！）